AI VIET NAM – AI COURSE 2025

Preprocessing Functions trong Tableau

(theo Data Type)

Tutorial

Dataset sử dụng trong ví dụ

Tableau Superstore — bộ dữ liệu mẫu có sẵn trong Tableau Desktop, tại menu **Help** → **Sample Workbooks** → **Superstore**. Dataset gồm ∼10.000 dòng đơn hàng bán lẻ với các trường về sản phẩm, khách hàng, doanh thu và lợi nhuận.

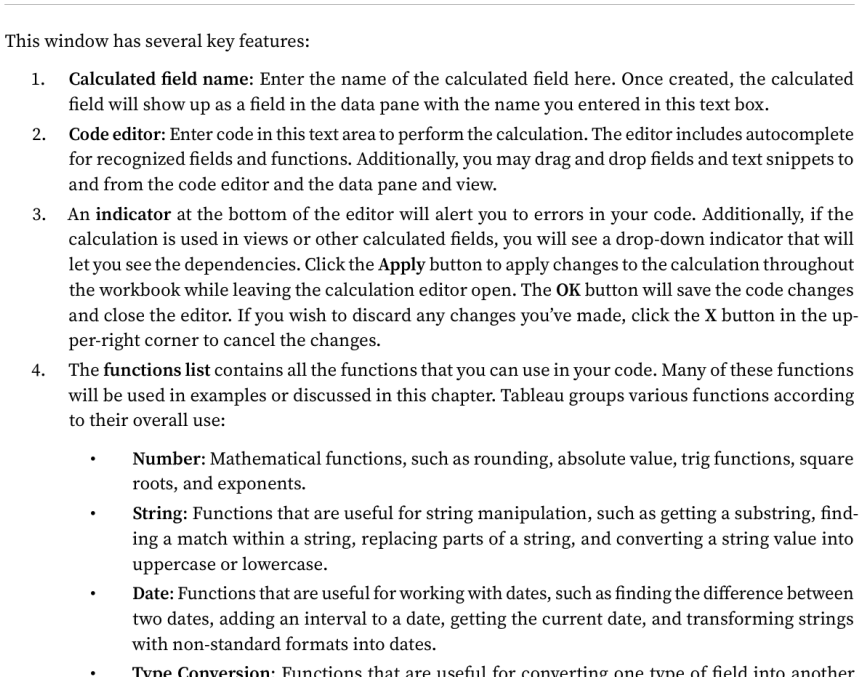
Mục lục

1 Giới thiệu — Dữ liệu thực tế không bao giờ "sạch" ngay từ đầu

Trong thực tế, hầu hết các bộ dữ liệu đều mang theo những vấn đề tiềm ẩn mà mắt thường khó nhận ra. Một file CSV xuất từ hệ thống ERP có thể chứa tên khách hàng bị viết không nhất quán (`"nguyen van a"` và `"NGUYEN VAN A"` là hai khách hàng khác nhau trong mắt Tableau), cột ngày tháng bị lưu dưới dạng văn bản thay vì kiểu Date thực sự, hay cột doanh thu bị nhận nhầm là String khiến Tableau từ chối tính tổng.
Thông thường, các lỗi này được xử lý trong Python với Pandas trước khi đưa vào Tableau. Tuy nhiên, Tableau cung cấp một công cụ mạnh mẽ ngay bên trong môi trường phân tích — **Calculated Field** — cho phép chúng ta biến đổi, chuẩn hóa và tạo cột mới mà không cần rời khỏi Tableau. Điều này đặc biệt hữu ích khi dữ liệu nguồn cập nhật thường xuyên và ta không muốn phải chạy lại pipeline xử lý mỗi lần.

Calculated Field (Trường tính toán)

Calculated Field là một cột dữ liệu mới được tạo ra bằng cách viết biểu thức (expression) trực tiếp trong Tableau. Calculated Field không thay đổi dữ liệu gốc trong nguồn — nó chỉ thực hiện phép tính tại thời điểm hiển thị, giống như một cột "ảo" được tính toán theo thời gian thực.
Cách tạo: Trên thanh menu, chọn **Analysis** → **Create Calculated Field...** Cửa sổ công thức sẽ hiện ra, nơi bạn nhập biểu thức và Tableau sẽ gợi ý hàm tự động.



This window has several key features:

1. **Calculated field name:** Enter the name of the calculated field here. Once created, the calculated field will show up as a field in the data pane with the name you entered in this text box.
2. **Code editor:** Enter code in this text area to perform the calculation. The editor includes autocomplete for recognized fields and functions. Additionally, you may drag and drop fields and text snippets to and from the code editor and the data pane and view.
3. An **indicator** at the bottom of the editor will alert you to errors in your code. Additionally, if the calculation is used in views or other calculated fields, you will see a drop-down indicator that will let you see the dependencies. Click the **Apply** button to apply changes to the calculation throughout the workbook while leaving the calculation editor open. The **OK** button will save the code changes and close the editor. If you wish to discard any changes you've made, click the **X** button in the upper-right corner to cancel the changes.
4. The **functions list** contains all the functions that you can use in your code. Many of these functions will be used in examples or discussed in this chapter. Tableau groups various functions according to their overall use:
   - **Number:** Mathematical functions, such as rounding, absolute value, trig functions, square roots, and exponents.
   - **String:** Functions that are useful for string manipulation, such as getting a substring, finding a match within a string, replacing parts of a string, and converting a string value into uppercase or lowercase.
   - **Date:** Functions that are useful for working with dates, such as finding the difference between two dates, adding an interval to a date, getting the current date, and transforming strings with non-standard formats into dates.
   - **Type Conversion:** Functions that are useful for converting one type of field into another

Hình 1: Cửa sổ Calculated Field — các thành phần chính gồm: (1) ô nhập tên field, (2) vùng soạn code, (3) indicator lỗi, và (4) danh sách hàm phân loại theo Number, String, Date, Type Conversion...

Trong phần này, chúng ta sẽ khám phá các hàm preprocessing được tổ chức theo **kiểu dữ liệu (data type)** — vì mỗi kiểu có bộ hàm riêng phù hợp với đặc điểm của nó.

2 Data Types trong Tableau

Khi kết nối với nguồn dữ liệu, Tableau tự động phân tích và gán **kiểu dữ liệu (data type)** cho từng cột dựa trên nội dung của nó. Mỗi data type được nhận diện thông qua một **icon đặc trưng** hiển thị bên trái tên field trong Data Pane. Việc hiểu đúng data type cực kỳ quan trọng vì nó quyết định hàm nào có thể áp dụng, loại aggregation nào được phép, và biểu đồ nào Tableau sẽ gợi ý khi bạn drag field vào canvas.
Tableau hỗ trợ 6 data type cơ bản, được tóm tắt trong bảng sau:

Icon	Data Type	Mô tả	Ví dụ trong Superstore
Abc	**String**	Văn bản — chuỗi ký tự bất kỳ	`Customer Name`, `Product Name`, `Region`
#	**Number**	Số — nguyên hoặc thập phân	`Sales`, `Profit`, `Quantity`, `Discount`
	Date	Ngày tháng — chỉ có ngày, không có giờ	`Order Date`, `Ship Date`
	Date & Time	Ngày tháng kèm giờ phút giây	Dấu thời gian giao dịch hệ thống
T|F	**Boolean**	Nhị phân — chỉ có True hoặc False	Kết quả so sánh, điều kiện lọc
	Geographic	Địa lý — Tableau tự nhận diện để vẽ bản đồ	`Country`, `State`, `City`, `Postal Code`

- **Number (decimal):** These are numeric values that include places after the decimal. Values such as `0.02`, `100.377`, or `3.14159` are decimal values.
- **Number (whole):** These are integer or whole numbers with no fractional values or places after the decimal. Values such as `5`, `157`, and `1,455,982` are whole numbers.
- **Date and Time:** These are dates along with times. Values such as `November 8, 1980 12:04:33 PM` are date and time types.
- **Date:** These are dates without times. Values such as `July 17, 1979` are date types.
- **String:** These are a series of characters. A string may consist of a mixture of alphabetic characters, numeric characters, symbols, or special characters. They may even be blank (empty). Values such as `Hello World`, `password123`, and `%$@*!` are all strings. In code, strings will be surrounded by single or double quotes.
- **Boolean:** This is a true or false value. The values `TRUE` and `FALSE`, and the expressions `1=1` (which evaluates as true) and `1=2` (which evaluates as false), are all Boolean types.
- **Spatial:** A complex value that describes a location, line, or shape as it relates to a spatial area.

Every field in Tableau has one of these data types and every function in Tableau returns one of these data types. Some functions expect input that matches some of these types, and you'll receive errors if you try to pass in the wrong type.

Some types can be converted into other types. For example, using some of the type conversion functions mentioned above, you could convert the string `"2.81"` into the decimal value `2.81`. You could convert that decimal value into a whole number, but in that case, you'd lose the places after the decimal value and the whole number would simply be `2`.

A data type is different from the format displayed. For example, you may choose to format a decimal as a percentage (for example, `0.2` could be shown as `20%`), as currency (for example, `144.56` could be formatted as `$144.56`), or even as a number with 0 decimals (for example, `2.81` would be rounded to `3`).

Pay attention to the data types of fields and functions as we continue.

Hình 2: Data Types trong Tableau — giải thích chi tiết các kiểu dữ liệu: Number (decimal và whole), Date and Time, Date, String, Boolean, và Spatial.

Lưu ý

Tableau có thể nhận sai data type. Tableau nhận diện data type dựa trên mẫu dữ liệu trong file, nhưng đôi khi mắc lỗi. Trường hợp phổ biến nhất là cột `Order Date` trong file CSV bị lưu với định dạng không chuẩn (`"15/1/2024"` thay vì `"2024-01-15"`) khiến Tableau nhận nhầm thành String — mọi hàm ngày tháng sẽ thất bại.
Cách kiểm tra nhanh: Nhìn vào icon bên trái tên field trong Data Pane. Nếu thấy **Abc** cho một cột mà bạn biết là ngày tháng hoặc số, đó là dấu hiệu dữ liệu cần được chuyển đổi kiểu trước khi phân tích.

3 Preprocessing Functions theo Data Type

Tableau cung cấp hàng chục hàm preprocessing, được phân loại theo data type mà chúng xử lý. Trong thực tế, phần lớn các vấn đề tiền xử lý dữ liệu đều rơi vào 4 nhóm: **String**, **Number**, **Date**, và **Type Conversion**. Chúng ta sẽ đi sâu vào từng nhóm với bối cảnh thực tế và ví dụ cụ thể từ dataset Superstore.

3.1 String Functions — Xử lý văn bản

Khi làm việc với dữ liệu thực tế, các trường văn bản thường là nguồn gốc của nhiều lỗi tinh vi nhất. Một cột `Customer Name` xuất từ hệ thống CRM có thể chứa khoảng trắng thừa ở đầu và cuối (do nhập liệu), viết hoa viết thường không nhất quán, hoặc thậm chí có ký tự đặc biệt không mong muốn. Những lỗi này không gây ra thông báo lỗi — Tableau vẫn chạy bình thường — nhưng kết quả phân tích sẽ sai: cùng một khách hàng nhưng bị đếm thành nhiều người khác nhau, filter không hoạt động đúng, hay group by ra kết quả thiếu.
Tableau cung cấp một bộ hàm String phong phú để giải quyết các vấn đề này. Dưới đây là các hàm thực dụng nhất mà một nhà phân tích dữ liệu cần nắm:

- **`UPPER(string)` và `LOWER(string)`:** Chuyển đổi toàn bộ chuỗi sang chữ HOA hoặc chữ thường. Đây là thao tác chuẩn hóa đầu tiên khi cần so sánh hoặc nhóm dữ liệu văn bản — `"Furniture"` và `"furniture"` là hai giá trị khác nhau trong Tableau, nhưng sau khi áp dụng `LOWER()` chúng sẽ hợp nhất thành một nhóm duy nhất.

- **`TRIM(string)`:** Xóa khoảng trắng ở **đầu và cuối** chuỗi — không xóa khoảng trắng ở giữa. Đây là một trong những hàm quan trọng nhất vì khoảng trắng thừa hoàn toàn vô hình khi nhìn bằng mắt, nhưng khiến `"Hà Nội"` và `" Hà Nội"` trở thành hai chuỗi khác nhau và filter sẽ không khớp.

- **`LEFT(string, number)` và `RIGHT(string, number)`:** Trích xuất `n` ký tự từ bên trái hoặc bên phải của chuỗi. Ứng dụng phổ biến: trích xuất mã tỉnh/thành từ `"HN-001"` bằng `LEFT([City Code], 2)` để lấy `"HN"`.

- **`MID(string, start, length)`:** Trích xuất một đoạn con từ giữa chuỗi, bắt đầu từ vị trí `start` (1-indexed) với độ dài `length` ký tự. Ví dụ `"US-2024-00123"` — ta có thể dùng `MID([Order ID], 4, 4)` để lấy năm `"2024"`.

- **`REPLACE(string, old, new)`:** Tìm và thay thế tất cả các lần xuất hiện của `old` bằng `new`. Ví dụ: `REPLACE([Phone], "-", "")` để xóa dấu gạch trong số điện thoại `"090-123-4567"` → `"0901234567"`.
- **`CONTAINS(string, substring)`:** Kiểm tra xem chuỗi có chứa chuỗi con không, trả về `True` hoặc `False`. Ví dụ: `CONTAINS([Product Name], "Chair")` trả về True cho tất cả sản phẩm có chữ "Chair" trong tên.
- **`LEN(string)`:** Đếm tổng số ký tự trong chuỗi. Thường dùng để kiểm tra dữ liệu: nếu mã sản phẩm phải có đúng 8 ký tự, `LEN([Product ID]) != 8` sẽ phát hiện các mã bị lỗi.
- **`SPLIT(string, delimiter, token_number)`:** Tách chuỗi thành các phần dựa trên ký tự phân cách và lấy phần thứ `token_number`. Ví dụ `SPLIT("Furniture/Bookcases", "/", 2)` trả về `"Bookcases"`.
- **Toán tử nối chuỗi +:** Dấu cộng `+` dùng để **ghép nối (concatenate)** các chuỗi. Ví dụ: `[First Name] + " " + [Last Name]` tạo ra `"Claire Gute"`. Lưu ý: **phải thêm khoảng trắng thủ công** vào công thức (`+ " " +`).
- **`STARTSWITH(string, substring)`:** Kiểm tra xem chuỗi có **bắt đầu** bằng chuỗi con không, trả về `True/False`. `STARTSWITH([Order ID], "US")` trả về True cho tất cả đơn hàng có mã bắt đầu bằng "US".
- **`ENDSWITH(string, substring)`:** Kiểm tra xem chuỗi có **kết thúc** bằng chuỗi con không. `ENDSWITH([File Name], ".csv")` trả về True cho các file CSV.
- **`FIND(string, substring, [start])`:** Tìm vị trí của chuỗi con trong chuỗi — trả về số thứ tự ký tự (1-indexed), hoặc **0 nếu không tìm thấy**. Ví dụ: `FIND("CA-2024-01", "-")` = 3.
- **`LTRIM(string)` và `RTRIM(string)`:** Xóa khoảng trắng **chỉ bên trái** (`LTRIM`) hoặc **chỉ bên phải** (`RTRIM`) — chi tiết hơn `TRIM()` vốn xóa cả 2 phía.

Ví dụ tổng hợp — Chuẩn hóa tên khách hàng:
Giả sử trường `Customer Name` trong Superstore có giá trị lộn xộn như `"  CLAIRE GUTE  "` (thừa khoảng trắng, toàn chữ hoa). Chúng ta tạo một Calculated Field mới tên `[Customer Name Cleaned]`:

```
// X a  k h o ng  t r ng  d  + c h u y n  v   c h  t h  Íng
          c h u n  h a so s nh
LOWER(TRIM([Customer Name]))

//  K ít  q u  : "claire gute"
```

Hoặc nếu muốn kiểm tra nhanh tên nào còn khoảng trắng thừa (để lọc ra dữ liệu lỗi):

```
// True = c   k h o ng  t r ng  t h a  Ỉ   u   h o c  c u i
LEN([Customer Name]) != LEN(TRIM([Customer Name]))

//  K ít  q u  : True / False
```

what if we have room numbers such as 1056 on the tenth floor, and 1617 on the sixteenth? We'd have to consider other options, such as the following:

```
MID([Room], 0, LEN([Room]) - 2)
```

Although this is more complicated, the string functions return a substring that starts at the beginning of the string and ends just before the last two characters. That gives us floor `5` for `506`, and floor `18` for `1856`.

We've now considered some row-level calculation examples. Let's move on to the next major type of calculation in Tableau: aggregate-level calculations.

Aggregate calculations

We've already considered aggregations such as `sum`, `min`, and `max` in Tableau. Often, you'll use fields as simple aggregations in the view. But sometimes, you'll want to use aggregations in more complex calculations.

For example, you might be curious to explore the percentage of the rent that was discounted. There is no such field in the data. It could not really be stored in the source, because the value changes based on the level of detail present in the view (for example, the discount percentage for an individual unit will be different from the discount percentage per floor or per building). Rather, it must be calculated as an aggregate and recalculated as the level of detail changes.

Hình 3: Ví dụ String Functions — sử dụng `MID` và `LEN` để trích xuất thông tin từ chuỗi. Mã màu cú pháp giúp phân biệt tên hàm (xanh), tên field (cam), và hằng số (tím).

Tableau không có hàm PROPER()

Python có `str.title()` và Excel có `PROPER()` để viết hoa chữ cái đầu mỗi từ, nhưng Tableau không có hàm tương đương sẵn có.
Workaround đơn giản cho tên một từ: `UPPER(LEFT([Name], 1)) + LOWER(MID([Name], 2, LEN([Name])))`
Tuy nhiên với tên nhiều từ (như tên người), workaround này chỉ viết hoa được từ đầu tiên. Đối với trường hợp này, tốt hơn nên xử lý bằng Python trước khi import vào Tableau.

3.2 Number Functions — Xử lý số

Dữ liệu số trong Tableau ít gặp lỗi về kiểu hơn String, nhưng vẫn cần xử lý trong nhiều tình huống: làm tròn để hiển thị gọn gàng, lấy trị tuyệt đối khi chỉ quan tâm độ lớn, hay xử lý giá trị NULL khiến mọi phép tính đều trả về NULL thay vì giá trị thực.
Trong dataset Superstore, trường `Profit` là ví dụ điển hình — nó có thể mang giá trị âm khi đơn hàng bị lỗ, và trường `Discount` thường là số thập phân (0.1, 0.2, ...) cần làm tròn khi hiển thị. Dưới đây là các hàm Number thực dụng nhất:

- **`ROUND(number, decimal_places)`:** Làm tròn đến số chữ số thập phân chỉ định. `ROUND([Sales], 2)` làm tròn doanh thu đến 2 chữ số thập phân. Khi `decimal_places = 0`, hàm làm tròn về số nguyên gần nhất. Khi `decimal_places` là số âm, hàm làm tròn về hàng chục, hàng trăm... ví dụ `ROUND(1234, -2)` = 1200.

- **`CEILING(number)` và `FLOOR(number)`:** Hai hàm làm tròn "lệch" — `CEILING()` luôn làm tròn **lên** đến số nguyên tiếp theo (ví dụ `CEILING(3.2)` = 4), còn `FLOOR()` luôn làm tròn **xuống** (ví dụ `FLOOR(3.8)` = 3). Hữu ích trong các bài toán phân nhóm hay làm tròn giá vé, chi phí vận chuyển.

- **`ABS(number)`:** Trả về giá trị tuyệt đối — bỏ dấu âm. `ABS([Profit])` cho biết độ lớn của lãi/lỗ bất kể chiều hướng. Thường dùng: `ABS([Actual] - [Target])`.

- **`ZN(expression)`:** Viết tắt của **"Zero if Null"** — nếu biểu thức trả về NULL thì thay bằng 0, ngược lại giữ nguyên giá trị. Đây là một trong những hàm quan trọng

nhất khi làm việc với dữ liệu có missing values, vì trong Tableau bất kỳ phép tính nào có NULL đều trả về NULL: `NULL + 100 = NULL`, `NULL * 5 = NULL`.

- **`POWER(number, exponent)`:** Tính lũy thừa. `POWER([Sales], 2)` = Sales bình phương. Cần thiết khi tính các chỉ số thống kê hoặc chuẩn hóa dữ liệu.
- **`SIGN(number)`:** Trả về **−1** nếu số âm, **0** nếu bằng 0, **1** nếu dương. Kết hợp với CASE: `CASE SIGN([Profit]) WHEN 1 THEN "Lãi" WHEN -1 THEN "Lỗ" ELSE "Hòa vốn" END`.
- **`DIV(integer1, integer2)`:** Chia nguyên — trả về phần nguyên của phép chia. `DIV(7, 2)` = 3 (không phải 3.5). Hữu ích khi tính số lô hàng nguyên: `DIV([Quantity], [Batch Size])`.

Ví dụ tổng hợp — Phân loại đơn hàng theo lợi nhuận:

```
// Ph n  l o i   n   h ng: L i ,  H u   v n ,  h o c   L
IF [Profit] > 0 THEN "Profitable"
ELSEIF [Profit] = 0 THEN "Break-even"
ELSE "Loss"
END

//  K ỉt   q u  : "Profitable" / "Break-even" / "Loss" cho   t ng
    d ng
```

Và tính **tỷ suất lợi nhuận (profit margin)** một cách an toàn, tránh lỗi chia cho 0:

```
// ZN    m     b o   khi Sales = NULL  t h    k ỉt   q u   = 0 thay
    v   NULL
ZN([Profit]) / ZN([Sales])

//  H o c   v i ỉt   c n   t h n  h  n        tr nh   l i   chia cho
    0:
IF ZN([Sales]) = 0 THEN 0
ELSE [Profit] / [Sales]
END
```

NULL trong Tableau ≠ 0

Đây là một trong những điểm gây nhầm lẫn phổ biến nhất. Trong Tableau, NULL không phải là 0 — nó là "không có dữ liệu". Mọi phép tính số học với NULL đều trả về NULL, không phải một số.
Ví dụ thực tế: Nếu một khách hàng chưa có đơn hàng nào trong tháng, trường `Sales` của họ là NULL (không phải 0). `SUM(NULL, 50, 30)` = 80 — hàm SUM bỏ qua NULL. Nhưng `NULL / 100` = NULL — phép chia thất bại. Luôn dùng `ZN()` trước khi thực hiện phép chia để đảm bảo an toàn.

3.3 Date Functions — Xử lý ngày tháng

Phân tích theo thời gian là một trong những yêu cầu phổ biến nhất trong Business Intelligence. Với dataset Superstore có hai trường date là `Order Date` và `Ship Date`, chúng ta cần khả năng: tách năm/quý/tháng để nhóm dữ liệu, tính khoảng cách giữa ngày đặt hàng và ngày giao hàng, hay lọc ra các đơn hàng trong 30 ngày gần nhất so với ngày hôm nay.
Trước khi bắt đầu, cần hiểu khái niệm quan trọng nhất trong Date Functions — tham số **`date_part`**. Đây là một chuỗi chỉ định **đơn vị thời gian** mà hàm sẽ làm việc với:

Giá trị `date_part`	**Ý nghĩa**	**Kết quả (ví dụ với 15/03/2024)**
`'year'`	Năm	2024
`'quarter'`	Quý trong năm	1 (Q1: tháng 1–3)
`'month'`	Tháng trong năm	3
`'day'`	Ngày trong tháng	15
`'weekday'`	Thứ trong tuần	6 (6 = Thứ Sáu, ISO)
`'hour'`	Giờ (chỉ dùng với Date & Time)	0–23

Các hàm Date thực dụng nhất trong phân tích dữ liệu bán hàng:

- **`YEAR(date)`, `MONTH(date)`, `DAY(date)`:** Ba hàm đơn giản nhất để trích xuất năm, tháng, ngày từ một trường Date. `YEAR([Order Date])` trả về số nguyên như 2021, 2022... Đây là cách nhanh nhất để tạo các trường phụ trợ phục vụ nhóm dữ liệu theo thời gian.

- **`DATEPART(date_part, date)`:** Tổng quát hơn `YEAR/MONTH/DAY` — có thể lấy bất kỳ đơn vị thời gian nào. `DATEPART('quarter', [Order Date])` trả về 1, 2, 3, hoặc 4. Kết quả là **số nguyên**, phù hợp cho tính toán.

- **`DATENAME(date_part, date)`:** Tương tự `DATEPART` nhưng trả về **tên dạng chuỗi**. `DATENAME('month', [Order Date])` trả về `"January"`, `"February"`... Hữu ích khi muốn hiển thị nhãn thân thiện trên biểu đồ.

- **`DATEDIFF(date_part, start_date, end_date)`:** Tính **khoảng cách** giữa hai ngày. `DATEDIFF('day', [Order Date], [Ship Date])` cho biết mất bao nhiêu ngày để giao hàng.

- **`DATEADD(date_part, interval, date)`:** Cộng (hoặc trừ nếu `interval` âm) một khoảng thời gian vào một ngày. `DATEADD('month', 3, [Order Date])` trả về ngày 3 tháng sau ngày đặt hàng.

- **`TODAY()`:** Trả về ngày hôm nay (không có giờ). Thường kết hợp với `DATEDIFF` để tạo filter động.

- **`DATETRUNC(date_part, date)`:** "Cắt" ngày về **đầu kỳ** tương ứng. `DATETRUNC('month', #2024-03-15#)` = **1/3/2024**; `DATETRUNC('quarter', #2024-03-15#)` = **1/1/2024**. Cực kỳ hữu ích khi cần so sánh các kỳ với nhau.

- **`NOW()`:** Trả về **ngày và giờ hiện tại** (Date & Time) — khác với `TODAY()` chỉ trả về ngày. Lưu ý: `NOW()` tính theo múi giờ của máy chủ Tableau, không phải browser người dùng.
- **`QUARTER(date)` và `WEEK(date)`:** Shorthand nhanh — `QUARTER([Order Date])` trả về 1–4; `WEEK([Order Date])` trả về số tuần trong năm (1–52/53).
- **`MAKEDATE(year, month, day)`:** Tạo một giá trị Date từ 3 số nguyên riêng lẻ. `MAKEDATE(2024, 3, 15)` tạo ngày 15/3/2024. Hữu ích khi dữ liệu lưu riêng năm, tháng, ngày trong 3 cột Number.
- **`DATEPARSE(format, string)`:** Chuyển đổi một **chuỗi** có định dạng đặc biệt thành kiểu Date. Ví dụ:

```
DATEPARSE("MMMM,d,yy", "September,4,12")
// K it   q u  : 9/4/2012 (ng y th ng n m theo ISO)
```

Các định dạng phổ biến: `yyyy` = năm 4 chữ số, `MM` = tháng 2 chữ số, `dd` = ngày 2 chữ số, `MMMM` = tên tháng đầy đủ, `MMM` = tên tháng viết tắt. Chuỗi format phải **khớp chính xác** với cấu trúc của chuỗi ngày.

Ví dụ tổng hợp — Phân tích thời gian giao hàng:

```
// T nh  s   ng y giao h ng ( t   Order Date   n   Ship Date
   )
DATEDIFF('day', [Order Date], [Ship Date])
// K it   q u  : 2, 3, 5... ( s   nguy n)

// Ph n  l o i   t c        giao h ng
IF DATEDIFF('day', [Order Date], [Ship Date]) <= 2
THEN "Fast"
ELSEIF DATEDIFF('day', [Order Date], [Ship Date]) <= 5
THEN "Normal (3-5 days)"
ELSE "Slow (>5 days)"
END

// L c    n   h ng trong 365 ng y  g n   n h t
DATEDIFF('day', [Order Date], TODAY()) <= 365
// K it   q u  : True / False     d ng l m filter condition
```

DATEPART vs DATENAME: khi nào dùng cái nào?

- Dùng **`DATEPART('month', ...)`** → kết quả là số `1, 2, ..., 12` → khi cần **tính toán** (so sánh, cộng trừ)
- Dùng **`DATENAME('month', ...)`** → kết quả là `"January", "February", ...` → khi cần **hiển thị nhãn** trên trục X

Lưu ý: kết quả của `DATENAME` sẽ sắp xếp theo bảng chữ cái (April, August, December...) thay vì theo thứ tự tháng. Để sắp xếp đúng, hãy dùng thêm `DATEPART` làm trường sắp xếp phụ trợ (Sort Field).

3.4 Ad-hoc Calculations và xử lý giá trị NULL

Ngoài việc tạo Calculated Field đầy đủ qua menu **Analysis → Create Calculated Field...**, Tableau còn hỗ trợ một cách tạo công thức nhanh hơn gọi là **Ad-hoc Calculation**. Phương pháp này cho phép nhập công thức **trực tiếp trên Columns shelf, Rows shelf, hoặc thẻ Marks** mà không cần đặt tên hay lưu lại.

Cách tạo Ad-hoc Calculation:

1. Kéo field `Profit` vào Columns shelf. Tableau hiển thị `SUM([Profit])`.
2. Nhấp đúp vào pill `SUM([Profit])` trên Columns shelf — một ô nhập công thức nhỏ xuất hiện.
3. Sửa thành `SUM([Profit])/SUM([Sales])` và nhấn **Enter**. Tableau tính và hiển thị kết quả ngay lập tức.
4. Nếu muốn lưu lại: kéo pill từ shelf vào **Data Pane**, Tableau sẽ yêu cầu đặt tên cho Calculated Field.

Tableau có hệ thống **mã màu cú pháp** trong Formula Editor:

Màu/kiểu chữ	Ý nghĩa
Chữ đỏ (gạch chân)	Lỗi cú pháp — di chuột lên để xem gợi ý sửa
`// chữ màu xám`	Comment — không ảnh hưởng tính toán
[**chữ màu cam**]	Tên field — cần bao bằng dấu `[]`
chữ màu xanh	Tên hàm (function)
[**chữ màu tím**]	Parameters
chữ in đậm	Phép tính được aggregate
chữ không in đậm	Phép tính ở mức row-level

Xử lý giá trị NULL với `ZN()`:

Khi kết hợp hoặc trộn lẫn nhiều nguồn dữ liệu, một số ô có thể không có giá trị (NULL). Trong Tableau, `NULL + bất kì số nào = NULL` — nghĩa là một giá trị NULL duy nhất sẽ làm cho toàn bộ kết quả là NULL.

Hàm `ZN(expression)` (Zero-if-Null) giải quyết vấn đề này:

```
// Kh ng an to n:  n ỉu  Discount l   NULL,  k ỉt   q u   s í
   l   NULL
[Sales] * [Discount]

// An to n: ZN thay NULL  b ng  0  t r   c
ZN([Sales]) * ZN([Discount])
```

Lưu ý: `ZN` chỉ dùng đối với số (Number) — thay NULL số bằng 0 thường hợp lý (không có Discount có nghĩa Discount = 0%), nhưng cần cân nhắc kỹ trong ngữ cảnh cụ thể.

3.5 Type Conversion Functions — Chuyển đổi kiểu dữ liệu

Một trong những vấn đề phổ biến nhất khi import dữ liệu vào Tableau là **kiểu dữ liệu bị nhận sai**. Nguyên nhân thường gặp: file CSV có cột số nhưng chứa ký tự không phải

số (dấu phẩy "1,234.56", đơn vị tiền tệ "$500"), cột ngày tháng có định dạng không chuẩn, hoặc Tableau đọc toàn bộ cột là String vì có một vài ô chứa chữ.
Khi dữ liệu bị nhận sai kiểu, hậu quả rất rõ ràng: không thể tính `SUM([Revenue])` nếu Revenue là String, không thể dùng `YEAR([Order Date])` nếu Order Date là String. Type Conversion Functions cho phép chúng ta ép kiểu (cast) dữ liệu sang đúng loại:

- **`INT(expression)`:** Chuyển đổi sang số nguyên, cắt bỏ phần thập phân (không làm tròn — `INT(3.9)` = 3, không phải 4).
- **`FLOAT(expression)`:** Chuyển đổi sang số thực (có phần thập phân). Phổ biến hơn `INT` vì giữ được độ chính xác. `FLOAT("1234.56")` trả về số `1234.56`.
- **`STR(expression)`:** Chuyển đổi **sang String**. Hữu ích khi cần ghép Number vào chuỗi: `"Order #" + STR([Order ID])` tạo ra `"Order #12345"`.
- **`DATE(expression)`:** Chuyển đổi String hoặc Number sang kiểu Date. `DATE("2024-01-15")` trả về ngày 15/1/2024 với đầy đủ khả năng sử dụng các Date Functions.
- **`DATETIME(expression)`:** Tương tự `DATE()` nhưng tạo ra giá trị Date & Time, bao gồm cả giờ phút giây.

Ví dụ tổng hợp — Sửa cột dữ liệu bị nhận sai kiểu:

```
// T r   Íng   h p   1:   C t   "Revenue_str" = "1234.56" (String)
// ->  C h u y n   th nh Number        c    t h   SUM, AVG
FLOAT([Revenue_str])
//  K ít   q u  : 1234.56

// T r   Íng   h p   2:   C t   "Order_Date_str" = "2024-01-15" (
   String)
// ->  C h u y n   th nh Date        d ng  YEAR(), DATEDIFF()...
DATE([Order_Date_str])
//  K ít   q u  : 15/01/2024 (Date)

// T r   Íng   h p   3:   T o   label  k ít   h p   s   v   c h
"  n   h ng #" + STR([Order ID]) + "     " + STR([Customer Name
   ])
//  K ít   q u  : "  n   h ng #12345     Claire Gute"
```

Lưu ý

Nhận biết lỗi kiểu dữ liệu ngay từ đầu — Có 3 dấu hiệu rõ ràng:
1. Field có icon **Abc** trong Data Pane nhưng bạn biết nó là số hoặc ngày
2. Tableau báo lỗi **"Cannot mix aggregate and non-aggregate arguments"** khi bạn cố tính SUM
3. Tableau không gợi ý **Line Chart** khi bạn drag một Date field vào Columns

Bước xử lý chuẩn: tạo Calculated Field mới dùng `FLOAT()`, `DATE()`, hay `INT()` để chuyển đổi, đặt tên rõ ràng (ví dụ `[Sales (Numeric)]`), rồi sử dụng field mới đó thay cho field gốc.

3.6 Logical Functions — Phân loại và điều kiện hóa dữ liệu

Nếu String Functions giúp làm sạch dữ liệu và Date Functions giúp phân tích thời gian, thì **Logical Functions** là trái tim của mọi phân tích dữ liệu có chiều sâu — chúng cho phép Tableau đưa ra **quyết định** dựa trên điều kiện, phân loại dữ liệu thành nhóm, và xử lý các trường hợp đặc biệt.

3.6.1 IF / ELSEIF / ELSE / END — Điều kiện nhiều nhánh

Cú pháp IF là cách phổ biến nhất để phân loại dữ liệu trong Tableau:

```
IF <test1> THEN <value1>
ELSEIF <test2> THEN <value2>
...
ELSE <default_value>
END
```

- Tableau kiểm tra **lần lượt** từng điều kiện, trả về giá trị tương ứng với điều kiện đầu tiên là `True`
- `ELSEIF` và `ELSE` là tùy chọn — có thể viết `IF...THEN...END` đơn giản
- Phần `ELSE` là giá trị mặc định khi không có điều kiện nào khớp — nếu bỏ qua, kết quả là `NULL`

Ví dụ thực tế — Phân loại lợi nhuận:

```
//  T o  Calculated Field [Profit Status]
IF [Profit] > 0 THEN "Profitable"
ELSEIF [Profit] = 0 THEN "Break-even"
ELSE "Loss"
END
```

Ví dụ nâng cao — Phân loại theo nhiều trường:

```
// Ph n  l o i  kh ch  h ng VIP  d a  tr n  c   Sales  v
   Profit
IF SUM([Sales]) > 10000 AND SUM([Profit]) > 1000
THEN "VIP"
ELSEIF SUM([Sales]) > 5000
THEN "Regular"
ELSE "New"
END

// L  u  í : Khi  d  ng  IF  k ỉt   h p  aggregate (SUM),
//  c   c ng  t h c  p h i  Ỉ   c p  aggregate
```

IIF (Inline IF)

`IIF(<test>, <then>, <else>)` là phiên bản rút gọn của IF cho trường hợp chỉ có 2 nhánh. `IIF([Profit] > 0, "Lãi", "Lỗ")` tương đương với `IF [Profit] > 0 THEN "Lãi" ELSE "Lỗ" END`. Dùng khi công thức đơn giản — nhưng khi cần nhiều điều kiện, IF/ELSEIF dễ đọc hơn nhiều IIF lồng nhau.

3.6.2 CASE / WHEN / THEN — Phân loại theo giá trị cụ thể

CASE là lựa chọn tốt hơn IF khi bạn cần so sánh **một field với nhiều giá trị cụ thể**:

```
CASE <expression>
WHEN <value1> THEN <result1>
WHEN <value2> THEN <result2>
...
[ELSE <default>]
END
```

Ví dụ thực tế — Dịch tên vùng sang tiếng Việt:

```
CASE [Region]
WHEN "East" THEN " M i n    ng  "
WHEN "West" THEN " M i n  T  y "
WHEN "North" THEN " M i n   B c "
WHEN "South" THEN " M i n  Nam"
ELSE "Kh ng  x  c    nh   "
END
```

IF vs CASE: khi nào dùng cái nào?

CASE	IF
So sánh **một field** với nhiều **giá trị rời rạc**	Kiểm tra **nhiều điều kiện phức tạp** khác nhau
`[Region] = "East"`, `[Region] = "West"`...	`[Sales] > 10000 AND [Profit] > 0`...
Cú pháp gọn hơn, dễ đọc hơn	Linh hoạt hơn — dùng được `>`, `<`, `AND`, `OR`

3.6.3 Toán tử logic: AND, OR, NOT

Các toán tử này kết hợp nhiều điều kiện vào một biểu thức:

- **`AND`:** Cả hai điều kiện đều phải `True`. `[Sales] > 1000 AND [Profit] > 0` — chỉ đơn hàng lãi VÀ có doanh thu cao. AND dùng **short-circuit evaluation**.
- **`OR`:** Chỉ cần một trong hai điều kiện là `True`. `[Region] = "East" OR [Region] = "West"`.

- **NOT:** Đảo ngược kết quả boolean. `NOT ISNULL([Customer ID])` — chỉ giữ lại các dòng có Customer ID.

```
// V    d    k ỉt   h p :   n    h ng   c n   ch   í
// ( L    l n   H O C   c h i ỉt   k h u   cao) V   kh ng   p h i
   kh ch VIP
(SUM([Profit]) < -500 OR SUM([Discount]) > 0.3)
AND NOT ([Customer Segment] = "VIP")
```

3.6.4 Xử lý NULL: ISNULL, IFNULL và ZN

Ba hàm xử lý NULL trong Tableau, mỗi hàm phục vụ mục đích khác nhau:

Hàm	Cú pháp	Trả về	Dùng khi
ISNULL	`ISNULL(expr)`	`True/False`	Cần **kiểm tra** xem field có NULL không
IFNULL	`IFNULL(e1, e2)`	Giá trị e1 hoặc e2	Cần **thay thế NULL bằng giá trị bất kỳ**
ZN	`ZN(expr)`	Giá trị hoặc **0**	Cần **thay NULL bằng 0** cho phép tính số

```
// ISNULL     P h  t   h i n   d    l i u   t h i ỉu
ISNULL([Customer ID])
//  K ỉt   q u  : True cho  c  c  d ng  ch a  c   Customer ID

// IFNULL     Thay NULL  b ng   g i    t r    m c     nh    c
   ngh a
IFNULL([Assigned Region], "Ch a  ph n  v ng")
//  K ỉt   q u  :  C h u i   t h c   h o c  "Ch a  ph n  v ng"

// ZN        m     b o  an to n cho  ph p  t nh  s
ZN([Discount]) * ZN([Sales])
//  K ỉt   q u  :  N ỉu  NULL ->  k ỉt   q u   l   0, kh ng  p h i
   NULL
```

Lưu ý

IFNULL vs ZN vs ISNULL:

- `ISNULL` chỉ **phát hiện** (trả Boolean), không thay thế
- `IFNULL` thay NULL bằng **bất kỳ giá trị nào** (String, Number, Date...)
- `ZN` chỉ thay bằng **0** — ngắn gọn nhưng chỉ dùng cho Number
- Với String: dùng `IFNULL([Field], "N/A")` thay vì ZN
- Với Number: có thể dùng cả `IFNULL([Field], 0)` hoặc `ZN([Field])`

3.6.5 IN — So sánh với tập giá trị

`IN` kiểm tra xem một giá trị có nằm trong một danh sách hay không — ngắn gọn hơn nhiều `OR` liên tiếp:

```
// Thay v :
[Region] = "East" OR [Region] = "West" OR [Region] = "Central"

// D ng IN:
[Region] IN ("East", "West", "Central")
// K it q u : True n iu Region b ng b t k gi t r
   n o trong danh s ch
```

`IN` cũng dùng được với Sets trong Tableau: `[Customer] IN [Top 10 Customers Set]`.

4 Bảng tổng hợp — Quick Reference

Bảng dưới đây tóm tắt toàn bộ các hàm đã học, sắp xếp theo vấn đề thực tế mà bạn có thể gặp:

Data Type	Vấn đề thực tế	Hàm xử lý	Ví dụ nhanh
String	Khoảng trắng thừa	`TRIM()`, `LTRIM()`, `RTRIM()`	`TRIM([Name])`
String	Hoa/thường không nhất quán	`UPPER()`, `LOWER()`	`LOWER([Region])`
String	Trích xuất một phần	`LEFT()`, `RIGHT()`, `MID()`	`LEFT([Code], 3)`
String	Thay thế ký tự lạ	`REPLACE()`	`REPLACE([Phone], "-", "")`
String	Kiểm tra chuỗi con	`CONTAINS()`, `STARTSWITH()`	`STARTSWITH([ID], "US")`
String	Tìm vị trí chuỗi con	`FIND()`	`FIND([Code], "-")`
Number	Làm tròn	`ROUND()`	`ROUND([Sales], 2)`
Number	Giá trị NULL	`ZN()`	`ZN([Profit])`
Number	Trị tuyệt đối	`ABS()`	`ABS([Profit])`
Number	Hướng âm/dương	`SIGN()`	`SIGN([Profit])`
Date	Nhóm theo năm/quý/tháng	`YEAR()`, `DATEPART()`	`QUARTER([Date])`
Date	Cắt về đầu kỳ	`DATETRUNC()`	`DATETRUNC('month', [Date])`
Date	Label thân thiện	`DATENAME()`	`DATENAME('month', [Date])`
Date	Khoảng cách 2 ngày	`DATEDIFF()`	`DATEDIFF('day', [S], [E])`
Date	Ngày giờ hiện tại	`NOW()`, `TODAY()`	`TODAY()`
Kiểu sai	String → Number	`FLOAT()`, `INT()`	`FLOAT([Sales_str])`
Kiểu sai	String → Date	`DATE()`, `DATEPARSE()`	`DATE([Date_str])`
Kiểu sai	Number → String	`STR()`	`"ID: " + STR([ID])`
Logic	Phân loại nhiều nhánh	`IF/ELSEIF/ELSE/END`	`IF [P]>0 THEN "Lãi" ...`
Logic	Phân loại theo giá trị	`CASE/WHEN/THEN/END`	`CASE [Region] WHEN ...`
Logic	Kiểm tra NULL	`ISNULL()`	`ISNULL([Customer ID])`
Logic	Thay NULL bằng default	`IFNULL()`, `ZN()`	`IFNULL([Region], "N/A")`

Phần tiếp theo: ***Basic Charts*** *— Tạo các loại biểu đồ cơ bản trong Tableau.*